Innovation Roadmap: Security Engine Features

This document outlines advanced, novel features that can elevate the project from a standard security scanner to a high-level research platform. These features target "grey areas" where traditional industry tools currently struggle.

---

1. Autonomous Agentic False-Positive Triage

Description: Uses an LLM to "double-check" scanner findings, filtering out noise before the developer sees it.

- **Difficulty:** Medium
- **Strategy:**
    1. Extract the code snippet around a finding (e.g., 5 lines above/below).
    2. Pass the snippet + the scanner's rule ID to the AI Security Agent.
    3. Prompt the agent to perform "Reachability Analysis" (is the input actually sanitized?).
    4. Update the Unified JSON with an `ai_confidence_score`.

2. Autonomous Proof-of-Concept (PoC) Generator

Description: Automatically generates a Python exploit script to prove that a detected vulnerability is actually "breakable."

- **Difficulty:** High
- **Strategy:**
    1. For high-confidence SAST findings (like SQLi or XSS), provide the AI Agent with the API endpoint and the vulnerable variable.
    2. Instruct the Agent to write a `requests`-based Python script that attempts to trigger the flaw.
    3. Include this script in the final report as a "Download PoC" button.

3. Live Secret Verification (Active Validation)

Description: Actively tests leaked secrets (API keys, Tokens) to see if they are live and dangerous or just dead strings.

- **Difficulty:** Low
- **Strategy:**
    1. Create a "Verification Library" of safe, read-only API calls for common services (AWS, GitHub, Slack, Stripe).
    2. When Gitleaks finds a secret, trigger the matching API call.

3. If the call succeeds (200 OK), escalate the severity to "URGENT - LIVE EXPLOIT."

4. Semantic PII (Privacy) Flow Analysis

Description: Tracks sensitive data (Passwords, Credit Cards, SSNs) through the code to ensure they aren't being printed to logs or sent to external trackers.

- **Difficulty:** Medium
- **Strategy:**
  1. Build a custom "Taint Analysis" rule using Opengrep that looks for variables named after PII (e.g., `email`, `phone`, `token`).
  2. Track if these "sources" ever end up in a "sink" (like `console.log` or a `requests.post` to a non-internal domain).

5. Temporal Risk & "Toxic" Code Lineage

Description: Calculates risk based on the history of the code (how many people edited it, how old the bug is).

- **Difficulty:** Low
- **Strategy:**
  1. When a bug is found on a specific line, execute `git blame --porcelain` on that line.
  2. Calculate "Code Churn" (how many times that file has changed in the last 30 days).
  3. Flag files with high churn and multiple authors as "Unstable Hotspots."

6. Supply Chain "Shadow" Health Monitor

Description: Predicts library risks before they become CVEs by analyzing the health of the open-source repository itself.

- **Difficulty:** Medium
- **Strategy:**
  1. For every package in `package.json` or `requirements.txt`, query the GitHub API.
  2. Check for "Red Flags": 0 commits in the last year, high ratio of open issues to PRs, or a sudden change in ownership.
  3. Assign a "Project Vitality Score" to each dependency.

7. AI-Specific (LLM-Sec) Scanning

Description: The world's first SAST engine specifically for detecting "Prompt Injection" and "Insecure Output Handling" in AI-integrated apps.

- **Difficulty:** High
- **Strategy:**
    1. Create custom rules to detect calls to LLM libraries (LangChain, OpenAI, etc.).
    2. Check if user-provided strings are being concatenated directly into prompts without delimiters.
    3. Flag code that passes LLM output directly to a `system` command or `eval()` call.

Implementation Recommendation:

To achieve the best "Wow" factor with the least effort, I recommend implementing **Feature 1 (AI Triage)** and **Feature 3 (Live Secret Verification)** first. These provide immediate, undeniable value that current open-source wrappers cannot match.